IT31 : Optimisation du Routage SDN (Programmation Dynamique & Parallélisme)

Date : 18 Avril 2026 **Objectif :** Résoudre l'explosion combinatoire $O(|V|!)$ du routage SDN sur les grandes topologies (Fat-Tree) en remplaçant la recherche exhaustive (DFS) par un algorithme de Plus Court Chemin (Dijkstra/Bellman) appliqué sur des tenseurs de poids dynamiques, couplé à une parallélisation du traitement des Workloads.

---

1. Formalisation Mathématique Étendue

1.1. Modèle Matriciel du Réseau

Soit $G = (V, E)$ le graphe du réseau.

- **Capacité $C \in \mathbb{R}^{|V| \times |V|}$** : $C_{i,j}$ est la bande passante physique du lien (i,j).
- **Trafic Alloué $\Lambda \in \mathbb{R}^{|V| \times |V|}$** : $\Lambda_{i,j}$ est la somme du trafic actif sur le lien (i,j).

1.2. Poids Dynamique (M/M/1)

Le temps de traversée d'un lien (i,j) dépend du délai de propagation statique $d_{i,j}$ et du délai de file d'attente M/M/1. Nous construisons une **Matrice de Poids Dynamiques $W(t)$** :

$$ W_{i,j}(t) = \begin{cases} d_{i,j} + \frac{1}{C_{i,j} - \Lambda_{i,j}(t)} & \text{si } C_{i,j} > \Lambda_{i,j}(t) \ \infty & \text{sinon (Congestion absolue)} \end{cases} $$

1.3. Programmation Dynamique

Plutôt que d'explorer tous les chemins entre la source S et la destination D, nous cherchons le chemin minimisant $\sum W_{i,j}$. Comme les poids $W_{i,j}$ sont toujours positifs, le problème se résout de manière exacte et optimale en $O(|V| \log |V|)$ via l'algorithme de **Dijkstra Dynamique** (ou Floyd-Warshall matriciel pour tous les couples).

$$ D(j) = \min_{i \in \text{Voisins}(j)} \left(D(i) + W_{i,j}(t) \right) $$

---

2. Implémentation Logicielle dans CloudSimSDN

Comment intégrer cette théorie dans le code Java de CloudSimSDN ?

Étape A : Dijkstra Dynamique au lieu du DFS (Problème du Routage)

Actuellement, la classe `NetworkOperatingSystem` utilise une méthode de type `getPaths(src, dst)` qui renvoie une liste *exhaustive* de tous les chemins possibles,

construite via DFS.

Solution :

1. Créer une nouvelle méthode `getOptimalDynamicPath(src, dst)` dans votre `LinkSelectionPolicyDynamicLatencyBw`.
2. Au lieu de demander tous les chemins, cette méthode lira l'état instantané du réseau (la matrice $W(t)$).
3. Elle exécutera un algorithme de Dijkstra "à la volée" sur ce graphe pondéré dynamiquement.
4. Elle ne retournera **que le meilleur chemin absolu**. *Gain : On passe d'un temps exponentiel (minutes/heures) à un temps quasi-instantané (millisecondes).*

Étape B : Parallélisation des Workloads (Mini-Batch)

CloudSimSDN traite les événements (Workloads) de manière purement séquentielle. Quand des centaines de workloads démarrent à $t = 0.0s$, ils sont routés un par un.

Solution (Le modèle Map-Reduce) :

1. **Regroupement (Batching) :** Dans `SDNController` (ou `SDNBroker`), au lieu d'envoyer les requêtes de routage une par une, on regroupe tous les workloads créés au même "tick" d'horloge du simulateur (ex: $t = 0.0$).
2. **Phase MAP (Parallèle) :**
   - On "gèle" la matrice des poids $W(t)$.
   - On utilise les Streams Parallèles Java 8 (`workloads.parallelStream()`).
   - Chaque thread exécute le Dijkstra Dynamique pour son workload de manière isolée et sans verrous (Mutex).
3. **Phase REDUCE (Fusion) :**
   - On récupère tous les chemins choisis par les threads.
   - On additionne la bande passante requise sur chaque lien réseau sollicité pour obtenir le Delta de trafic : $\Delta\Lambda$.
4. **Phase UPDATE (Séquentielle) :**
   - On met à jour l'état du réseau d'un seul coup : $\Lambda(t+1) = \Lambda(t) + \Delta\Lambda$.
   - Le prochain "Batch" de workloads (à $t = 0.1s$) utilisera cette nouvelle matrice de trafic.

---

3. Plan d'Action (Comment le coder)

Pour mettre en œuvre ceci, voici les classes Java exactes à modifier :

1. **`org.cloudbus.cloudsim.sdn.routing.DijkstraAlgorithm`** (ou création d'un `DynamicDijkstra.java`)
   - *Action :* Adapter l'algorithme classique pour qu'il n'utilise pas la latence statique des liens, mais qu'il interroge dynamiquement la formule M/M/1 à chaque itération du graphe.

2. **`LinkSelectionPolicyDynamicLatencyBw.java`**

    - *Action :* Supprimer la boucle `for (Path p : allPaths)` qui parcourait l'infinité de chemins. Remplacer par un appel direct : `Path best = dynamicDijkstra.getShortestPath(src, dst, currentTrafficMap);`

3. **`SDNController.java`**

    - *Action :* Modifier la méthode de traitement des requêtes (`processPacketRoute` ou `processWorkload`). Intercepter les requêtes simultanées, les stocker dans une `List<Workload>`, puis invoquer un `ExecutorService` ou `parallelStream()` pour demander les chemins simultanément.

Conclusion

Cette architecture (Dijkstra sur Graphe de Poids Dynamique + Map-Reduce des trafics) est exactement l'approche utilisée dans les contrôleurs SDN de production (ex: ONOS, OpenDaylight) pour garantir le passage à l'échelle (Scalability).